

18 Dynamische Speicherzuweisung



Bislang wurde stets mit Variablen gearbeitet, deren Struktur und Größe bereits während des Compilierens bekannt waren. Es gibt jedoch viele Situationen, bei denen nicht vorausgesagt werden kann, wie viel Speicherplatz gebraucht werden wird, so zum Beispiel, wenn eine (beliebig große) Datei eingelesen werden soll.

Die Sprache C bietet Mechanismen an, welche es erlauben, zur Laufzeit des Programmes dynamisch eine variable Menge an Bytes zu reservieren und wieder freizugeben. Diese sogenannten **dynamischen Speicherblöcke** oder Speicherbereiche befinden sich nicht wie die üblichen Variablen im Daten- oder Stack-Segment, sondern im sogenannten Heap-Segment eines Programmes (siehe auch Kapitel [→ 15.2.3](#)).

Der **Heap** ist ein Segment eines Programms neben den Segmenten Code, Daten und Stack. Aufgabe des Heaps ist es, Speicherplatz für die Schaffung dynamischer Speicherblöcke bereitzuhalten.



Dynamische Speicherblöcke werden in C bei Bedarf zur Laufzeit des Programms durch Aufruf der entsprechenden Bibliotheksfunktionen mit Hilfe des Laufzeitsystems auf dem Heap angelegt.



Die Verwaltung des Heaps erfolgt durch eine Komponente im Laufzeitsystem, die auch als Heap-Verwaltung bezeichnet wird. Die Heap-Verwaltung entscheidet beispielsweise, an welcher Stelle des Heaps Speicherplatz reserviert wird. Die Funktionen, die vom Programm aufgerufen werden, um dynamische Speicherblöcke anzulegen, geben dem Programm einen Pointer auf den im Heap reservierten Speicherbereich zurück.

Dieser Pointer kann in einer Variablen mit dem passenden Pointer-Typ gespeichert werden. Über diesen Pointer kann man dann auf die reservierte Speicherstelle im Heap zugreifen. In C kann durch Aufruf einer entsprechenden Bibliotheksfunktion ein Speicherblock auch wieder freigegeben werden. Dynamische Speicherblöcke stehen somit dem Programm von ihrer Erzeugung bis zu ihrer Freigabe zur Verfügung.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_18.

Die **Gültigkeit** und **Lebensdauer** eines dynamischen Speicherblockes wird nicht durch Blockgrenzen bestimmt, das heißt nicht durch die statische Struktur des Programms.



In C erfolgt die Reservierung von Speicherblöcken auf dem Heap beispielsweise mit den Bibliotheksfunktionen `malloc()` beziehungsweise `calloc()`, die Freigabe erfolgt mit der Bibliotheksfunktion `free()`.

Je nach System ist die Größe des Heaps stark begrenzt. Moderne Computer besitzen mehr als ausreichend Speicherplatz, dennoch sollte er nicht verschwendet werden. Wenn ständig nur Speicher angefordert und kein Speicher zurückgegeben wird, kann es zu einem Überlauf des Heaps kommen. Hinzu kommt das Problem, dass mit zunehmendem Gebrauch der Heap mehr und mehr zerstückelt wird, so dass es sein kann, dass keine größeren Speicherobjekte mehr angelegt werden können, obwohl in der Summe genügend freier Speicher vorhanden ist, aber eben nicht am Stück.

In Sprachen wie Java oder Smalltalk werden Speicherobjekte im Heap nicht explizit freigegeben. Ein Garbage Collector gibt hier Speicherplatz, der nicht mehr referenziert wird, wieder frei. Der Garbage Collector wird in unregelmäßigen Abständen durch das Laufzeitsystem aufgerufen. Er ordnet ferner den Speicher neu, so dass größere homogene unbenutzte Speicherbereiche auf dem Heap wiederhergestellt werden. In C gibt es keinen Garbage Collector.

18.1 Reservierung von Speicher

Dieses Kapitel beschreibt die verschiedenen Funktionen, um Speicher zu reservieren und wieder freizugeben.

Alle Funktionen befinden sich in der Bibliothek `<stdlib.h>`.

18.1.1 Die Funktion `malloc()`

Die Funktion **`malloc()`** dient zum dynamischen Reservieren eines Speicherobjekts im Heap.



Syntax:

```
void* malloc(size_t size);
```

In C kann die Reservierung von Speicher im Heap mit der Funktion `malloc()` erfolgen. Der Name `malloc()` kommt von „memory allocation“. Ein solchermaßen erzeugtes Speicherobjekt ist bis zum Programmende oder bis zur Rückgabe des Speichers mit `free()` gültig.

Im Gegensatz zu Java und C++, wo es jeweils eine Funktion `new()` gibt, welche Speicherobjekte des gewünschten Datentyps anlegt, ist die Sichtweise in C nicht so abstrakt. Die Funktion `malloc()` muss wissen, wie viele Bytes benötigt werden. Die Zahl der Bytes wird üblicherweise mit Hilfe des `sizeof`-Operators bestimmt. Beispielsweise:

```
MyObject* obj = malloc(sizeof(MyObject));
```

Wenn die Speicherreservierung durchgeführt werden konnte, gibt die Funktion `malloc()` einen Pointer auf den reservierten Speicherbereich zurück, ansonsten wird `NULL` zurückgegeben.

Der zurückgegebene Pointer ist vom Typ `void*` und wird bei einer Zuweisung implizit in den Typ des Pointers auf der linken Seite der Zuweisung gewandelt. Es sei daran erinnert, dass abgesehen von `void*` kein Pointer auf einen Datentyp an einen Pointer auf einen anderen Datentyp ohne explizite Umwandlung zugewiesen werden kann (siehe Kapitel [→ 8.2](#)).

Es empfiehlt sich, bei Zuweisungen mit `malloc()` stets einen expliziten Cast zu verwenden, damit der Compiler die Zuweisung auf Typsicherheit prüfen kann.

Hier noch ein Beispiel:

`malloc.c`

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int* pointer = (int*)malloc(sizeof(int));
    if (pointer!= NULL) {
        *pointer = 3;
        printf("Wert an Pointer-Adresse: %d\n", *pointer);
        free(pointer);
    } else {
        printf("Nicht genugend Speicher verfuegbar.\n");
    }
    return 0;
}
```

Hier die Ausgabe des Programms:

Inhalt des Pointers: 3

18.1.2 Die Funktion `aligned_alloc()`

Die Funktion `aligned_alloc()` existiert seit dem C11-Standard und erlaubt im Gegensatz zur Funktion `malloc()` nicht nur die gewünschte Größe des Speicherblocks anzugeben, sondern zusätzlich dessen Alignment (Anordnung) im Speicher. Der Prototyp für die Funktion `aligned_alloc()` ist der Folgende:

```
void* aligned_alloc(size_t alignment, size_t size);
```

Weitere Informationen zum Alignieren (Anordnen) von Speicherbereichen können im Kapitel [→ 9.9.1](#) nachgelesen werden.

18.1.3 Die Funktion `calloc()`

Die Funktion `calloc()` reserviert ebenfalls Speicher im Heap, allerdings eine bestimmte Anzahl von dynamischen Speicherblöcken. Außerdem führt sie eine Initialisierung durch.



Syntax:

```
void calloc(size_t num, size_t size);
```

Als Übergabeparameter erwartet die Funktion `calloc()` zwei Parameter. Als erster Parameter wird die Anzahl der benötigten Variablen erwartet. Der zweite Parameter entspricht der Größe einer einzelnen Variablen in Bytes. Damit eignet sich `calloc()` sehr gut zur Reservierung von Speicher für Arrays.

Im Gegensatz zu `malloc()` initialisiert `calloc()` den bereitgestellten Speicher und setzt dabei alle Bits auf 0. Daher rührt auch der Name: `calloc()` bedeutet „clear allocation“.



Hiebei ist wichtig zu wissen, dass die Funktion `free()` Speicherplatz zwar freigibt, ihn jedoch nicht initialisiert. Mittels `calloc()` wird somit eine irrtümliche Weiterverwendung von zuvor mit `free()` freigegebenen Daten vermieden. Siehe dazu Kapitel [→ 18.2](#).

Die Funktion `calloc()` gibt genauso wie `malloc()` einen Pointer auf den reservierten Speicherbereich zurück, wenn die Speicherreservierung durchgeführt werden konnte, ansonsten wird `NULL` zurückgegeben. Der Pointer ist vom Typ `void*` und wird bei einer Zuweisung implizit in den Typ des Pointers auf der linken Seite der Zuweisung gewandelt.

Hier wieder ein Beispiel:

calloc.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int* a = calloc(3, sizeof(int));
    if (a != NULL) {
        printf("Inhalt des Arrays: %d, %d, %d\n", a[0], a[1], a[2]);
        a[0] = 1;
        a[1] = 2;
        a[2] = 3;
        printf("Inhalt des Arrays: %d, %d, %d\n", a[0], a[1], a[2]);
        free(a);
    } else {
        printf("Nicht genug Speicher verfuegbar.\n");
    }
    return 0;
}
```

Hier die Ausgabe des Programms:

```
Inhalt des Arrays: 0, 0, 0
Inhalt des Arrays: 1, 2, 3
```

18.1.4 Die Funktion `realloc()`

Mit `realloc()` kann man die Größe eines dynamischen Speicherbereichs ändern.



Syntax:

```
void* realloc(void* memblock, size_t size);
```

Um einen mit `malloc()` oder `calloc()` erzeugten dynamischen Speicherbereich auch nachträglich noch in seiner Größe ändern zu können, existiert die Funktion `realloc()`. Diese Funktion bekommt einen Pointer auf einen bereits existierenden dynamischen Speicherbereich übergeben und zusätzlich noch die Größe des gewünschten neuen Speicherbereiches.

Ist die angeforderte neue Anzahl Bytes größer wie bisher, so probiert die Funktion, den angeforderten Speicher noch an die bisherige Adresse anzuhängen. Ist dies nicht möglich, dann verschiebt `realloc()` den vorhandenen Speicherbereich an eine Stelle im Speicher, an der noch genügend Speicher frei ist.

Die Funktion `realloc()` kann aufgrund der Verschiebung von Speicher sehr aufwändig sein.



Ist die angeforderte neue Anzahl Bytes kleiner wie bisher, so wird der Speicherbereich nach der gewünschten Anzahl freigegeben. Dieser Speicherbereich darf danach ausgehend von der Adresse in `memblock` nicht mehr angesprochen werden.

Speicher, welcher aufgrund von `realloc()` freigegeben wird, wird genauso wie bei `free()` nicht initialisiert (siehe Kapitel [→ 18.2](#)). Der Inhalt bleibt bestehen.



Die Funktion `realloc()` gibt einen Pointer auf den reservierten Speicherbereich zurück, wenn die Speicherreservierung durchgeführt werden konnte, ansonsten wird `NULL` zurückgegeben. Der Pointer ist vom Typ `void*` und wird bei einer Zuweisung implizit in den Typ des Pointers auf der linken Seite gewandelt.

Übergibt man an `realloc()` den `NULL`-Pointer als Adresse auf einen vorhandenen dynamischen Speicherbereich, dann arbeitet `realloc()` wie `malloc()` und gibt einen Pointer auf einen neu erstellten dynamischen Speicherbereich zurück.

Hier wieder ein Beispiel:

realloc.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int* a = malloc(sizeof(int));
    *a = 3;
    printf("Inhalt nach malloc(): %d\n", *a);

    a = realloc(a, 2 * (sizeof(int)));
    a[1] = 6;
    printf("Inhalt nach realloc(): %d, %d\n", a[0], a[1]);

    free(a);
    return 0;
}
```

Hier die Ausgabe des Programms:

```
Inhalt nach malloc(): 3
Inhalt nach realloc(): 3, 6
```

Man beachte, dass in diesem Beispiel der Übersichtlichkeit wegen auf eine Prüfung des zurückgegebenen Pointers auf `NULL` weggelassen wurde.

18.2 Freigabe von Speicher mittels free()

Mit **free()** kann man einen dynamischen Speicherbereich freigeben.



Syntax:

```
void free(void* pointer);
```

In C muss man den nicht mehr benötigten Speicher, der mit den Funktionen `malloc()`, `calloc()` oder `realloc()` beschafft wurde, mit Hilfe der Funktion `free()` wieder freigeben. Der entsprechende Speicherbereich kann dann von der Heapverwaltung erneut vergeben werden.

Wenn der Funktion `free()` ein `NULL`-Pointer übergeben wird, passiert nichts.



Der formale Parameter von `free()` ist vom Typ `void*`. Damit kann ein Pointer auf einen beliebigen Datentyp übergeben werden. Die Funktion `free()` ist mit dem Pointer, den `malloc()`, `calloc()` oder `realloc()` geliefert hat, aufzurufen.

Wird die Funktion `free()` mit einem Pointer aufgerufen, der nicht zuvor mittels `malloc()` oder einer ähnlichen Funktion reserviert wurde, so ist das Ergebnis undefiniert.



Nach dem Aufruf von `free()` darf über den Pointer nicht mehr auf das Speicherobjekt zugegriffen werden, da er auf kein gültiges Speicherobjekt mehr zeigt.



Es ist aber selbstverständlich möglich, erneut `malloc()`, `calloc()` oder `realloc()` aufzurufen und den Rückgabewert von `malloc()`, `calloc()` oder `realloc()` dem vorher schon benutzten Pointer wieder zuzuweisen.

Das Löschen eines Speicherbereiches mittels free() bewirkt nicht das Löschen der Inhalte desselben. Sollte beispielsweise ein späterer Aufruf von malloc() wieder auf dieselbe Adresse zeigen, so sind die vorherigen Inhalte immer noch abrufbar.



Wenn reservierter Speicher nicht mehr benötigt wird, es jedoch vergessen geht, ihn mittels free() wieder freizugeben, so bleibt dieser Speicherbereich bis zum Programmende unbrauchbar. Man spricht dann von einem sogenannten „memory leak“, was im nächsten Unterkapitel noch genauer erläutert wird.

Beispiel:

free.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    int* intPointer = malloc(sizeof(int));

    *intPointer = 42;
    printf("Inhalt der Variablen: %d\n", *intPointer);

    // Memory Leak:
    intPointer = malloc(sizeof(int));
    // Nicht initialisierter Wert:
    printf("Inhalt der Variablen: %d\n", *intPointer);

    *intPointer = 101;
    printf("Inhalt der Variablen: %d\n", *intPointer);

    free(intPointer);
    return 0;
}
```

Hier die Ausgabe des Programms:

```
Inhalt der Variablen: 42
Inhalt der Variablen: 0
Inhalt der Variablen: 101
```

Hier wird der Pointer vor Ausgabe der zweiten Zeile überschrieben, ohne dass der alte Pointer freigegeben wird. Der alte Wert ist somit verloren. Gleichzeitig wird der neu zugewiesene Speicherplatz nicht initialisiert. Das Resultat der zweiten Zeile könnte somit auch komplett anders aussehen, denn der Inhalt der Variablen ist zu diesem Zeitpunkt unbestimmt, sprich es könnten irgendwelche Werte ausgegeben werden.

18.2.1 Speicherlecks

Wenn nicht daran gedacht wird, beanspruchten Speicher wieder freizugeben, entstehen sogenannte **Speicherlecks**, auf Englisch „**memory leaks**“ genannt.

Während der Laufzeit eines Programmes bleiben diese Speicherbereiche unerreichbar. Erst beim Programmende erfolgt automatisch die Freigabe von jeglichem noch reserviertem Speicher durch das Betriebssystem.

Bei kleineren Systeme, die ja gerade oft in C implementiert werden, wie zum Beispiel Robotersteuerungen, müssen Speicherlecks zwingend vermieden werden. Dort können aufgrund der knappen Ressourcen mit der Zeit sich die unbrauchbaren Speicherbereiche aufkumulieren und das System zum Erliegen bringen. Der Speicher kann nur durch einen sogenannten „Kaltstart“ des Gerätes wieder vollständig zurückgesetzt werden. Auch Programme auf großen Betriebssystemen können sich zu wahren Speicherfressern entwickeln, wenn nicht daran gedacht wird, angeforderten, aber mittlerweile nicht mehr benötigten Speicher wieder freizugeben.

Memory leaks treten normalerweise auf, wenn der Aufruf der Funktion free() schlicht vergessen geht. Sie können aber beispielsweise auch passieren, wenn Pointer-Variablen wiederverwendet werden, sprich der letzte Pointer, welcher die Adresse des reservierten Speicherbereiches speichert, überschrieben wird, ohne vorher den Speicher freizugeben.

In der markierten Zeile des Beispiels in Kapitel → 18.2 passiert genau das: Die Variable `intPointer` speichert eine noch gültige Adresse, wird jedoch durch die erneute Zuweisung mittels `malloc()` überschrieben. Das bisherige Speichersegment ist somit nicht mehr adressierbar und bleibt bis zum Programmende ungenutzt.

Bei Sprachen, die einen Garbage Collector vorsehen, werden solche nicht referenzierten Speicherobjekte vom Garbage Collector ermittelt und dem zur Verfügung stehenden freien Speicher zugeordnet.

Speicheranforderung und Speicherfreigabe verhalten sich wie eine öffnende und eine schließende Klammer, die stets paarig sein müssen. Während Compiler unpaarige Klammern entdecken können, hängen unpaarige Speicheranforderungen und Speicherfreigaben vom jeweiligen Programmablauf ab und können daher nicht von einem Compiler gefunden werden.

Man sollte es sich zur Angewohnheit machen, bei jedem `malloc()` beziehungsweise bei jedem `calloc()` sofort zu planen, wann und wo der angeforderte Speicher wieder freigegeben wird. Oft ist es nicht möglich, dies sofort zu codieren. Dann muss man gewissenhaft eine ToDo-Liste führen und ToDo-Kommentare in den Code einfügen, damit nicht mit der Zeit immer mehr potentielle Speicherlöcher entstehen, die äußerst schwer im Nachhinein zu entdecken und zu beseitigen sind.

18.3 Dynamisch erzeugte Arrays

In der Praxis ist zur Compilierzeit in den meisten Fällen nicht bekannt, wie viele Objekte einer Struktur gebraucht werden. Auch die Länge von Strings ist von vornherein meist schwer zu schätzen. Manchmal wird der Ansatz genommen, einfach zur Compile-Zeit ein sehr großes Array mit beispielsweise 10000 Komponenten anzulegen und darauf zu hoffen, dass dies für alle Zeiten genug ist.

Dies führt jedoch zu ineffizienten Programmen, deren Laufzeitverhalten und Speicherbedarf zu wünschen übrig lässt. In diesem Abschnitt soll daher gezeigt werden, wie man zur Laufzeit ein Array dynamisch auf dem Heap erzeugt.

Mit `malloc()` oder `calloc()` kann zur Laufzeit ein **dynamisches Array** einer aktuell berechneten Größe angelegt werden.



An dieser Stelle ein einfaches Beispiel:

einArray.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int anzahl = 3;
    int* ptrArray;

    ptrArray = malloc(anzahl * sizeof(int));
    for (size_t index = 0; index < anzahl; ++index) {
        ptrArray[index] = (int)index;
        printf("malloc Array[%d]: %d\n", (int)index, ptrArray[index]);
    }
    free(ptrArray);

    ptrArray = calloc(anzahl, sizeof(int));
    for (size_t index = 0; index < anzahl; ++index) {
        ptrArray[index] = (int)index;
        printf("calloc Array[%d]: %d\n", (int)index, ptrArray[index]);
    }
    free(ptrArray);

    return 0;
}
```

Hier eine mögliche Ausgabe des Programms:

```
malloc Array[0]: 0
malloc Array[1]: 1
malloc Array[2]: 2
calloc Array[0]: 0
calloc Array[1]: 1
calloc Array[2]: 2
```

In diesem Beispiel wurde erneut auf die Prüfung des Rückgabewertes von `malloc()` beziehungsweise `calloc()` verzichtet. Heutige Betriebssysteme ermöglichen Applikationen ohne große Mühen die Nutzung von Megabytes und Gigabytes an Speicher, weswegen ein Test auf fehlenden Speicher häufig vergessen wird. Wenn heutzutage die Funktion `malloc()` jemals fehlschlägt, dann hat das Programm sehr wahrscheinlich ein anderes, ernsthafteres Problem, als dass angeblich zu wenig Speicherplatz zur Verfügung stünde. Auf älteren oder kleineren Geräten jedoch sollte dieser Test nicht fehlen.

18.4 Übungsaufgaben

Aufgabe 1: Speichern eines Datenobjektes

Betrachten Sie folgende Definition:

```
typedef struct Person Person;  
struct Person {  
    char name[20];  
    int alter;  
};
```

- a) Schreiben Sie eine Funktion, welche den Inhalt einer solchen Struktur auf dem Bildschirm ausgeben kann. Die Funktion soll als einzigen Parameter einen Pointer auf die Struktur erhalten.

```
void printPerson(const Person* p);
```

- b) Schreiben Sie eine Funktion mit den Parametern `name` und `alter`, welche genügend Speicher für eine solche Struktur reserviert, sie mit den Werten korrekt befüllt und die Adresse zurückgibt. Nutzen Sie die Funktion `strncpy()`, um den Namen zu kopieren.

```
Person* allocatePerson(const char* name, int alter);
```

- c) Erstellen Sie ein Programm, welches mehrere dieser Strukturen anhand der gegebenen Funktionen erstellt und sie auf dem Bildschirm ausgibt. Danach soll der Speicher wieder gelöscht werden.
- d) Ändern Sie die Deklaration des Feldes `name` von `char[20]` auf den Typ `char*`. Stellen Sie sicher, dass beim Erstellen der Struktur genügend Speicherplatz reserviert wird, um den gesamten Namen zu speichern. Wie löschen Sie diesen Speicher wieder?
- e) Erstellen Sie eine Funktion, die alleinig dafür zuständig ist, eine als Pointer gegebene Struktur `Person` wieder geordnet aus dem Speicher zu löschen.

```
void deallocatePerson(Person* p);
```

- f) Schreiben sie zusätzlich eine Funktion, welche einen Pointer auf eine solche Struktur erwartet und sie mit den zusätzlichen Parametern name und alter befüllt. Rufen Sie diese Funktion von allocatePerson() aus auf.

```
void initPerson(Person* p, const char* name, int alter);
```

- g) Erstellen Sie ein ganzes Array von solchen Strukturen als Pointer:

```
Person* persons[COUNT];
```

Ändern Sie alle Funktionen so ab, dass mittels for-Schleifen die Allokation, das Ausdrucken und die Deallokation stattfindet.

Aufgabe 2: Eigener String mit realloc()

Betrachten Sie folgende Struktur:

```
typedef struct MyString MyString;
struct MyString {
    char* ptr;
    size_t byteSize;
};
```

- a) Schreiben Sie eine Funktion, welche eine solche Struktur zurückgibt, bei welcher der Pointer ptr auf einen reservierten Speicherblock zeigt mit der gewünschten Byteanzahl und dem gegebenen String.

```
MyString makeString(const char* str);
```

Vergessen Sie nicht das abschließende Null-Byte! Beachten Sie auch, dass hier nicht ein Pointer zurückgegeben wird.

Geben Sie den Inhalt des Pointers zur Kontrolle auf dem Bildschirm aus.

- b) Schreiben Sie eine weitere Funktion, welche eine solche Struktur als Pointer erwartet und sie so verändert, dass sie nach der Funktion die gewünschte neue Anzahl an Bytes speichern kann. Der Inhalt des Pointers ptr soll dabei so gut es geht erhalten bleiben. Nutzen Sie dafür die Funktion realloc().

```
void enlargeString(MyString* string, size_t byteSize);
```

- c) Schreiben genau dieselbe Funktion nochmals, aber ohne die Funktion realloc().